ENGINEERING DOCUMENTATION & MASTER GUIDE

DevFolio MERN

Comprehensive System Design, Architecture Flowcharts, Core Working Mechanics & Complete 360° Technical Interview Preparation Manual

ARCHITECTURE PATTERN

Decoupled Client-Server MERN Architecture with RESTful APIs & Stateless JWT Session Control

FRONTEND CORE

React 19, Vite, React Router 7, Tailwind CSS v4, DaisyUI v5, Context API, Client Canvas Engine

BACKEND & STORAGE

Node.js, Express 5 REST API, MongoDB Atlas, Mongoose ODM, Bcrypt Password Hashing

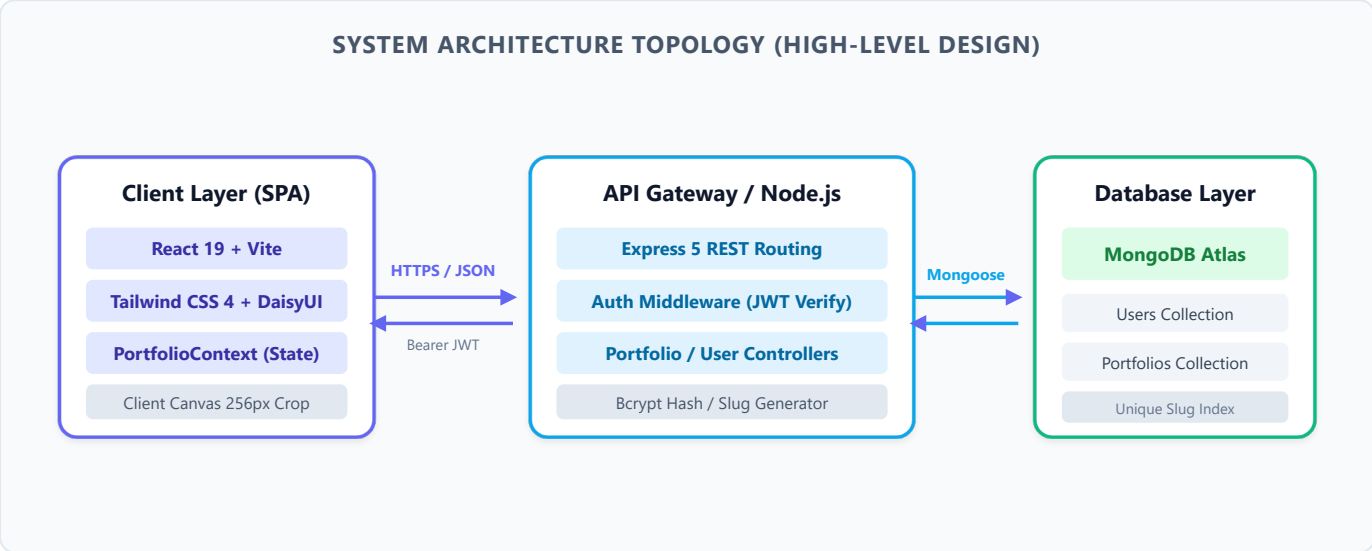
INTERVIEW PREPARATION

30+ In-Depth Questions spanning High Level Design, Concurrency, Security, Scalability & Code

Repository: github.com/ridhi431/mernprac **Author:** DevFolio Core Team **License:** MIT Open Source **Updated:** October 2026

1. System Overview & High-Level Architecture

DevFolio is an end-to-end full-stack portfolio builder built using the modern **MERN** stack (MongoDB, Express.js, React 19, Node.js). It empowers developers to build, customize, preview, and host professional portfolio websites in under two minutes with zero coding required on the viewer's part.

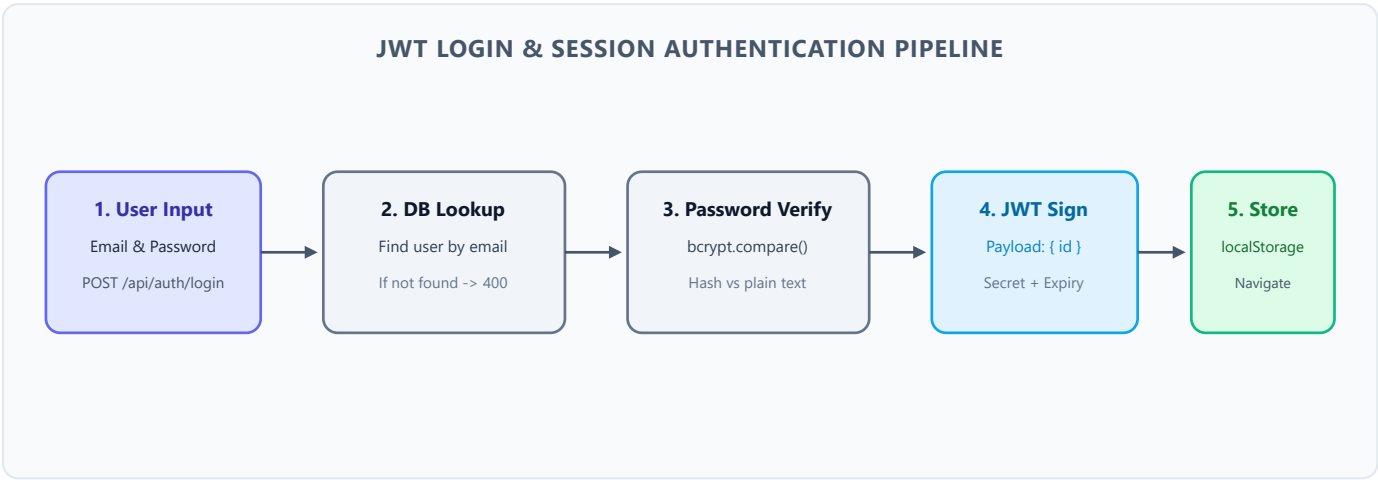


Key Architectural Principles

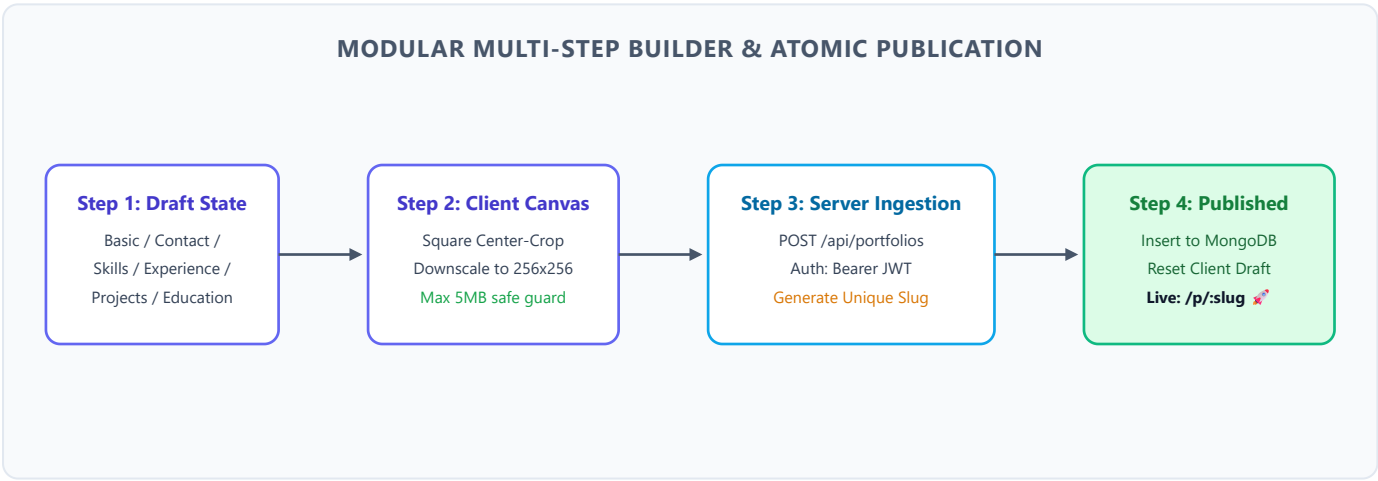
- **Stateless Authentication:** No session state is held on the Node server. Requests pass a cryptographically signed JWT via `Authorization: Bearer <token>`, allowing seamless horizontal autoscaling.
- **Client-Side Media Processing:** Profile photos are cropped to square and downscaled to 256x256 JPEG (quality 0.8) directly in the browser via HTML5 Canvas before network transmission, reducing payload size by 95% and eliminating server-side memory spikes.
- **Atomic Subdocument Persistence:** Projects, Skills, Experiences, and Education are modeled as nested arrays within a single Portfolio document, allowing single-roundtrip atomic read and update operations without heavy database joins.

2. Technical Flowcharts & Execution Sequences

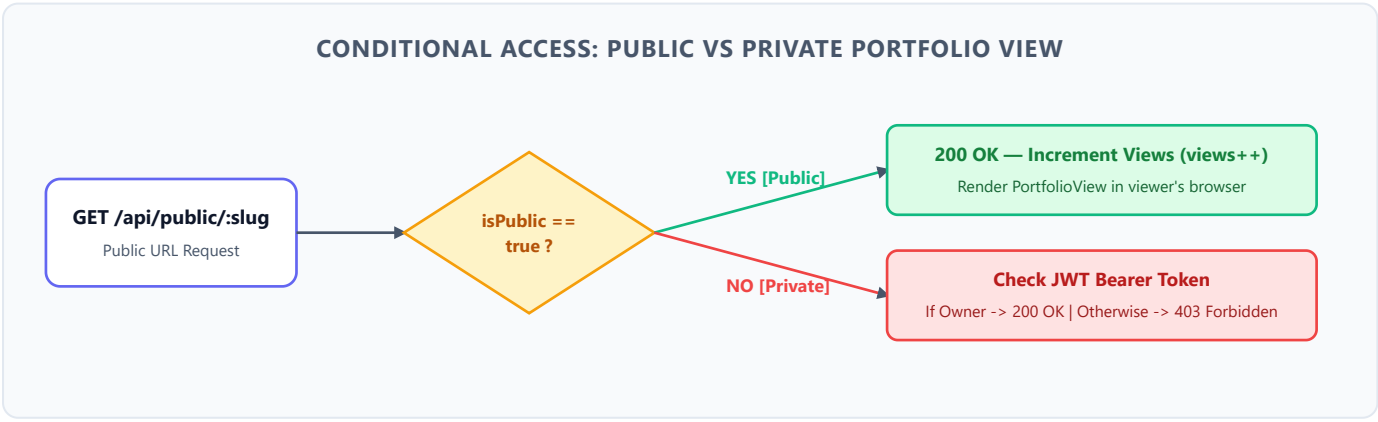
Flowchart 2.1: Authentication & Session Issuance



Flowchart 2.2: Portfolio Creation & Publishing Pipeline



Flowchart 2.3: Public Slug Resolution & Security Access Gate



3. Database Schema Design & Data Modeling

DevFolio utilizes **MongoDB** with **Mongoose** for document modeling. The data model is optimized for high-read throughput on public portfolio endpoints and fast nested updates on draft forms.

3.1 User Schema (`models/User.js`)

Field	Type	Constraints	Description & Purpose
`name`	String	Required, Trimmed	Full name of the user/creator
`email`	String	Required, Unique, Lowercase	Primary login identifier & unique account index
`password`	String	Required, Min length 6	One-way hashed password via Bcrypt (salt rounds = 10)
`photo`	String	Optional Base64 / URL	Profile avatar image (canvas downscaled to 256x256)
`bio`	String	Optional, Max 500 chars	Short description or developer summary
`timestamps`	Date	createdAt, updatedAt	Automatic audit timestamps managed by Mongoose

3.2 Portfolio Schema (`models/Portfolio.js`)

Field	Type	Index & Constraints	Description
`user`	ObjectId	Ref: 'User', Required, Indexed	Foreign key linking portfolio to its creator
`slug`	String	Unique, Required, Indexed	Human-readable URL slug (e.g. `/p/alex-fullstack-dev-7a8f`)
`theme`	String	Default: 'light'	DaisyUI theme identifier (17+ themes supported)
`isPublic`	Boolean	Default: true	Privacy toggle controlling global visibility
`views`	Number	Default: 0	Aggregated page view counter, incremented per public visit
`basic`	Object	Subdocument	`{ name, title, bio, photo }`
`contact`	Object	Subdocument	`{ email, github, linkedin }`
`skills`	[String]	Array of Strings	List of technical skills and tools
`projects`	[Object]	Array of Subdocuments	`[{ title, description, link }]`
`experience`	[Object]	Array of Subdocuments	`[{ company, role, duration, description }]`
`education`	[Object]	Array of Subdocuments	`[{ institute, degree, year }]`

> **Design Tradeoff Rationale:** We embedded `projects` and `experience` directly inside the Portfolio document instead of creating separate collections with relational references. Since a portfolio document is almost always read as a single atomic unit, embedding eliminates costly `$lookup` joins, cuts database roundtrips to 1, and enables fast caching.

4. Core Working & Engineering Implementations

4.1 Client-Side Canvas Image Downscaling Engine

Rather than uploading raw multi-megabyte photos to the Node server, DevFolio crops and compresses the photo inside an HTML5 Canvas element in the browser.

```
const resizeImage = (file, size = 256) =>
  new Promise((resolve, reject) => {
    const reader = new FileReader();
    reader.onload = () => {
      const img = new Image();
      img.onload = () => {
        const canvas = document.createElement('canvas');
        canvas.width = size;
        canvas.height = size;
        const min = Math.min(img.width, img.height);
        const sx = (img.width - min) / 2;
        const sy = (img.height - min) / 2;
        const ctx = canvas.getContext('2d');
        ctx.drawImage(img, sx, sy, min, min, 0, 0, size, size);
        resolve(canvas.toDataURL('image/jpeg', 0.8)); // 80% JPEG quality
      };
      img.src = reader.result;
    };
    reader.readAsDataURL(file);
  });
```

4.2 Mobile Responsiveness & Viewport Stabilization

- **Zero Horizontal Sway:** Enforced `overflow-x: hidden` and `max-width: 100vw` on `html, body, #root` to prevent mobile wobble from floating animated blobs.
- **Universal Drawer Architecture:** Synchronized DaisyUI mobile drawer across both `Mainlayout` and `Dashboardlayout` with auto-close navigation on link tap.
- **Touch-Device 3D Tilt Suppression:** Used `window.matchMedia('(hover: none)')` to prevent 3D perspective cards from freezing at awkward angles during touch swipes.

4.3 Complete REST API Reference

Method	Endpoint	Auth Level	Description
`POST`	`/api/auth/signup`	Public	Registers user, hashes password, returns JWT
`POST`	`/api/auth/login`	Public	Validates credentials, returns JWT
`GET`	`/api/users/me`	Private (Bearer)	Fetches logged-in user profile & photo
`PUT`	`/api/users/profile`	Private (Bearer)	Updates name, bio, and profile photo
`GET`	`/api/portfolios`	Private (Bearer)	Retrieves all portfolios belonging to user
`POST`	`/api/portfolios`	Private (Bearer)	Creates & publishes a new portfolio
`GET`	`/api/portfolios/:id`	Private (Owner)	Retrieves single portfolio by MongoDB ID for edit

Method	Endpoint	Auth Level	Description
`PUT`	`/api/portfolios/:id`	Private (Owner)	Updates existing portfolio
`DELETE`	`/api/portfolios/:id`	Private (Owner)	Deletes portfolio permanently
`GET`	`/api/portfolios/public/:slug`	Public / Conditional	Resolves public portfolio view & increments views

5. Master Technical Interview Questions & Answers

This section covers questions frequently asked by recruiters, tech leads, and hiring managers during full-stack MERN, system design, and frontend/backend deep-dive interviews.

Category A: Core Architecture & Framework Choices

MERN ARCHITECTURE **Q1: Walk me through the high-level architecture of DevFolio. Why did you choose this stack?**

Answer: DevFolio is structured as a decoupled Single Page Application (SPA) frontend with a RESTful backend API.
• **Frontend:** React 19 with Vite provides sub-second HMR and instant page renders. Tailwind CSS v4 and DaisyUI handle design and 17+ dynamic themes at runtime.
• **Backend:** Node.js with Express 5 manages API routing, authentication, and validation.
• **Database:** MongoDB with Mongoose manages polymorphic, schema-less portfolio documents.
Why this stack? The flexibility of JSON documents matches portfolio structures perfectly (different users have different sections: some have 10 projects, some have none). A decoupled architecture also allows us to host the frontend on a CDN (Vercel) for instant global delivery while running the API server independently on Node runtime.

STATE MANAGEMENT **Q2: Why did you use React Context API instead of Redux or Zustand for draft portfolios?**

Answer: Redux introduces substantial boilerplate (actions, reducers, store configuration) that is unjustified for our state scope. DevFolio has two localized domains of state:
1. `PortfolioContext`: Manages the active multi-step draft, section updates, and draft resetting.
2. `SettingsContext`: Manages global DaisyUI theme switching and localization.
React Context API paired with custom hooks (`usePortfolio()`, `useSettings()`) gives clean, idiomatic state sharing without external bundle overhead. For draft persistence, state is mirrored to `localStorage`, preventing data loss on accidental browser refreshes.

AUTHENTICATION **Q3: How does JWT authentication work in DevFolio? What are the tradeoffs of storing it in localStorage vs httpOnly Cookies?**

Answer: When a user logs in, the backend signs a JSON Web Token containing the user's MongoDB ID using a secret key and an expiration period (e.g. 7 days). The frontend stores this token and attaches it to subsequent requests in the `Authorization: Bearer <token>` header via an Axios interceptor/request config.
• **localStorage Tradeoff:** Highly convenient for client-side SPAs, easy to inspect and manage in pure client code. However, it is accessible via JavaScript, making it susceptible to Cross-Site Scripting (XSS) if unsanitized user inputs are rendered as HTML.
• **httpOnly Cookies:** Immune to XSS reading, but require CSRF protection tokens and custom CORS cookie-credentials setup across cross-domain deployments.
• **Mitigation in DevFolio:** We prevent XSS by exclusively using React's JSX auto-escaping (no `dangerouslySetInnerHTML`) and strictly sanitizing data inputs.

DATABASE MODELING **Q4: Why did you embed projects and experiences into the Portfolio document instead of normalizing them into separate collections?**

Answer: In relational design (3NF), you would create `projects`, `skills`, and `education` tables with foreign keys. In MongoDB, document embedding is preferred when:
1. *Contains-Relationship:* A project only belongs to that specific portfolio.
2. *Atomic Access Pattern:* Whenever a viewer loads `/p/:slug`, they need the basic info, projects, skills, and experience all together.
Embedding guarantees that a single `findOne({ slug })` fetch retrieves the complete portfolio in **O(1)** index time with zero joins (`$lookup`) and zero multi-table locking.

6. System Design, Scalability & Concurrency Questions

SYSTEM DESIGN **Q5: DevFolio suddenly goes viral and receives 10,000,000 portfolio views per day. What breaks first, and how would you scale the architecture?**

Answer:
1. Immediate Bottleneck: The MongoDB database will be overwhelmed by read traffic on `GET /api/portfolios/public/:slug` and atomic write contention on `views++`.
2. Architectural Solution (Multi-Tier Caching):
• **Edge CDN Caching:** Cache public portfolio HTML/JSON at Cloudflare/Vercel Edge with a TTL (e.g., 5 minutes) using Stale-While-Revalidate headers. 95% of reads never hit our servers.
• **Redis In-Memory Cache:** Store published portfolio JSON in Redis with key `portfolio:slug:{slug}`. On cache miss, fetch from MongoDB and warm Redis.
• **Decoupled View Counting (Write Buffer):** Instead of updating MongoDB on every single page view, increment an in-memory Redis counter (`INCR portfolio:views:{id}`). A background cron job flushes accumulated views to MongoDB in batch every 60 seconds.
• **Stateless Horizontal Autoscaling:** Run Node.js behind an AWS ALB (Application Load Balancer) in a Kubernetes cluster or AWS ECS with autoscaling based on CPU/Request count.

CONCURRENCY **Q6: How does DevFolio ensure slug uniqueness when two users publish simultaneously?**

Answer:
1. **Database-Enforced Uniqueness:** The `slug` field in the Mongoose schema has `unique: true` and an underlying B-tree unique index in MongoDB.
2. **Slug Generation Algorithm:** The slug is created by slugifying the user's name combined with a collision-resistant random alphanumeric hash (e.g. `alex-sharma-7b89x`).
3. **Optimistic Retry Loop:** If two requests attempt to insert the same slug concurrently, MongoDB throws a `E11000 duplicate key error`. The backend controller catches this specific error code, appends an additional random entropy suffix, and retries the save automatically.

CLIENT OPTIMIZATION **Q7: What was the motivation behind the Client-Side Canvas Image Processor?**

Answer: Uploading raw phone camera photos (often 5MB - 15MB) causes multiple severe issues:
• Network bottleneck on slow mobile connections (long upload times).
• Server-side memory spikes if Node.js buffers multiple 10MB images simultaneously.
• Exceeding MongoDB's 16MB per-document BSON limit.
Solution: We implemented a zero-dependency HTML5 Canvas engine. When a file is selected, it calculates the center square crop coordinates, downscales the image to exactly 256x256 pixels, and encodes it into a `image/jpeg` at 80% quality. The resulting payload is under 30KB—a 98% reduction in network payload with imperceptible visual difference for avatar rendering.

API DESIGN **Q8: How does the public slug resolution work, and how do you protect private portfolios from unauthorized viewing?**

Answer: The public endpoint is `GET /api/portfolios/public/:slug`:
1. Finds portfolio via `Portfolio.findOne({ slug })` using the indexed slug.
2. If `portfolio.isPublic === true`, access is granted to anyone, and view count increments.
3. If `portfolio.isPublic === false`, the server inspects the request `Authorization` header. If a valid JWT is present AND the token's user ID matches `portfolio.user.toString()`, access is granted (allowing the owner to preview their private portfolio).
4. If the token is missing or belongs to another user, it returns a strict `403 Forbidden ("This portfolio is private")`.

7. Security, Frontend & Real-World Bug Scenarios

Category B: Security & Vulnerability Protection

SECURITY **Q9: How do you protect passwords in DevFolio? Why is bcrypt salt rounds = 10 the sweet spot?**

Answer: Passwords are never stored in plain text. We use `bcryptjs` which incorporates a cryptographic salt and an iterative key derivation function.
• **Salt:** Random bits generated per password, preventing Rainbow Table attacks and ensuring two identical passwords have completely different hashes.
• **Work Factor (10 rounds):** Salt rounds represent \(2^{10} = 1024\) hashing iterations. This takes ~70ms on modern CPUs—completely negligible for a human logging in once, but computationally impossible for a brute-force attacker testing millions of combinations per second.

SECURITY **Q10: What is NoSQL Injection in MongoDB and how does DevFolio protect against it?**

Answer: NoSQL injection occurs when unvalidated user input containing MongoDB operators (like `{"$gt": ""}`) is passed directly into queries, allowing attackers to bypass authentication.
Defense in DevFolio:
1. *Mongoose Schema Typing:* Fields are strictly typed (e.g. `email: { type: String }`). If an attacker sends an object `{"$gt": ""}` where a String is expected, Mongoose casts or rejects it.
2. *Input Sanitization:* Explicit field destructuring (`const { email, password } = req.body`) ensures unexpected operator objects are not passed directly to database queries.

Category C: Mobile Responsiveness & Real-World Debugging

FRONTEND ENGINEERING **Q11: Tell me about a tricky mobile responsiveness bug you encountered in DevFolio and how you debugged and fixed it.**

Answer:
Problem: On mobile screens (iPhone SE / Samsung Galaxy), the landing page was swaying horizontally and the CTA buttons ("Create Portfolio" & "Go to Dashboard") had an uneven 24px left margin while touching the right edge with 0px margin.
Root Cause Analysis:
1. DaisyUI's `.hero` component utilizes a CSS Grid with `place-items: center` and internal padding on `.hero-content`.
2. When `w-full` was applied to child buttons, the width calculation combined with padding exceeded 100vw, pushing the content to the right.
3. Additionally, absolute positioned background animated blobs (`.pf-blob` with `filter: blur(80px)`) were shifting 30px outside the viewport boundary during animation, causing document horizontal expansion.
Solution:
• Replaced the conflicting CSS grid with a clean Flexbox layout and constrained button widths to `w-full max-w-xs sm:max-w-md mx-auto`, ensuring mathematical symmetry.
• Added `overflow-x: hidden` and `max-width: 100vw` on `html, body, #root` and applied `pointer-events: none` to all background blobs.

REACT 19 & STYLING **Q12: How does dynamic theme switching work in DevFolio without page reload?**

Answer: DevFolio uses DaisyUI's semantic CSS color variables (e.g. `var(--color-primary)`, `var(--color-base-100)`). In `PortfolioPreview` or `PortfolioView`, when a user selects a new theme from the dropdown, the theme name is passed via `data-theme={theme}` on the container wrapper, or set on `document.documentElement.setAttribute('data-theme', theme)`.
Because the colors are linked via CSS Custom Properties, the browser re-evaluates the CSS variables in GPU memory instantly within a single frame (16ms) without touching the DOM tree or re-rendering React components.

Category D: Behavioral & Future Roadmap

BEHAVIORAL **Q13: If you had another 2-3 weeks to work on DevFolio, what high-impact features would you add?**

Answer:
1. **Custom Domain Support:** Allow developers to point their custom CNAME (e.g., `portfolio.alex.dev`) to DevFolio using Cloudflare for SaaS or Vercel Domain API.
2. **One-Click PDF Resume Export:** Add a headless print or React-PDF generator to export the portfolio data into an ATS-friendly 1-page PDF resume.
3. **Granular Analytics:** Track unique visitor referrers, geographic distribution, and most-clicked project links using an embedded privacy-friendly event pipeline.
4. **AI-Powered Bio & Project Copywriter:** Integrate Google Gemini API to polish and generate compelling project summaries and headline elevator pitches from bullet points.

Interview Tip: When answering interview questions about DevFolio, always highlight *measurable engineering decisions*: reducing image payloads by 98% with Canvas, achieving O(1) single-document reads by embedding subdocuments, and guaranteeing zero mobile horizontal sway with strict viewport clamping.